Labo 16 — Praktijklabo: budget, diagnose, rechten en scheidingswanden

Doel: de API deployen met gedeclareerde resources, lezen wat de node ermee doet, een `OOMKilled` en een `Pending` uitlokken, een LimitRange en een quota zetten, een container herstarten en zijn vorige logs lezen, debuggen zonder shell in de image, een serviceaccount enkel leesrecht geven, en ten slotte het netwerk van de API sluiten voor alles behalve de frontend.

Vereisten — Labo 15 afgerond: k3s, de image `localhost/api:1.0` geïmporteerd in containerd (labo 12).

Meegeleverde bestanden (`files/`) - `deployment-api.yaml` — de API met `requests` / `limits` en haar Service. - `pod-oom.yaml` — een Pod die 100 MB toewijst met een limit van 32 Mi. - `pod-greedy.yaml` — een Pod die 64 Gi vraagt. - `limitrange.yaml` — een LimitRange en een ResourceQuota voor de namespace. - `rbac-reader.yaml` — ServiceAccount `reader`, Role `pod-reader`, RoleBinding. - `netpol-api.yaml` — NetworkPolicy: alleen Pods met `app=web` bereiken de API.

Stap 1 — Requests, limits en wat de node ermee doet

```
mkdir -p ~/labo-docker/16 && cd ~/labo-docker/16 && cp <labo-pad>/files/* .
kubectl create namespace lab16 ; kubectl config set-context --current --namespace=lab16
kubectl apply -f deployment-api.yaml ; kubectl rollout status deployment/api --timeout=60s
kubectl get pods -o custom-
columns=NAME:.metadata.name,QOS:.status.qosClass,REQ:.spec.containers[0].resources.requests.mem
ory,LIM:.spec.containers[0].resources.limits.memory
kubectl describe node | sed -n '/Allocated resources/,/Events/p' | head -9
```

Observeer twee Pods `Burstable` met `64Mi` / `256Mi`; en in de node `cpu 400m (2%)` aan requests en `memory 268Mi (0%)`: je twee Pods (200m, 128Mi) **plus** die van `kube-system`.

```
sleep 30 ; kubectl top nodes ; kubectl top pods
```

Observeer `wsl-ubuntu 35m 0% 789Mi 2%` (jouw waarden verschillen) en, voor elke Pod van de API, enkele millicores en `22Mi` werkelijk geheugen: ruim onder de request van 64 Mi, heel ver van de limit. Lees je `metrics not available yet`, wacht dan nog 30 s: metrics-server bemonstert om de 15 s.

Uitleg. De tabel `Allocated resources` telt de **requests** (wat de scheduler gereserveerd heeft); `kubectl top` toont het **werkelijke verbruik**. Die twee vallen nooit samen, en het is het verschil ertussen dat je afstelt.

Stap 2 — `OOMKilled` en `Pending`

```
kubectl apply -f pod-oom.yaml ; sleep 15 ; kubectl get pod glouton
kubectl describe pod glouton | grep -E 'Reason:|Exit Code|Restart Count'
kubectl get pod glouton -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}' ; echo
```

Observeer `glouton 0/1 OOMKilled 1 (14s ago)` (of al `CrashLoopBackOff`); `Reason: OOMKilled` en `Exit Code: 137` **twee keer** (de huidige en de vorige toestand: de herstarte container is op dezelfde manier gestorven), `Restart Count: 1`; en `OOMKilled` in `lastState`.

```
kubectl apply -f pod-greedy.yaml ; sleep 5 ; kubectl get pod trop-gourmand
kubectl describe pod trop-gourmand | grep FailedScheduling | cut -c1-150
kubectl delete pod glouton trop-gourmand --now
```

Observeer `trop-gourmand 0/1 Pending` en het event `Warning FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`.

Uitleg. De limit wordt toegepast door de cgroup: 100 MB in 32 Mi, de kernel doodt, de kubelet herstart. De request wordt toegepast door de scheduler: 64 Gi op een node die ze niet heeft, niemand plaatst, niets start.

Stap 3 — LimitRange en ResourceQuota

```
kubectl apply -f limitrange.yaml
kubectl run sans-limites --image=docker.io/library/alpine --command -- sleep 3600 ; sleep 2
kubectl get pod sans-limites -o jsonpath='{.spec.containers[0].resources}' ; echo
kubectl describe quota budget | sed -n '/Resource/,$p'
```

Observeer een Pod die zonder `resources` gemaakt is en er toch heeft: `{"limits":{"cpu":"250m","memory":"128Mi"},"requests":{"cpu":"50m","memory":"32Mi"}}`; en de quota: `limits.memory 640Mi / 2Gi`, `pods 3 / 10`, `requests.cpu 250m / 2`, `requests.memory 160Mi / 1Gi`.

```
kubectl run enorme --image=docker.io/library/alpine --overrides='{"spec":{"containers":[{"name":"e","image":"docker.io/library/alpine","command":["sleep","1"],"resources":{"limits":{"memory":"4Gi"}}}]}}' 2>&1 | cut -c1-160
kubectl delete pod sans-limites --now
```

Observeer `Error from server (Forbidden): pods "enorme" is forbidden: maximum memory usage per Container is 1Gi, but limit is 4Gi`.

Uitleg. De LimitRange **vult** Pods zonder resources **aan** (standaardwaarden) en **weigert** die welke `max` overschrijden; de quota telt alles wat in de namespace leeft. Beide werken bij de creatie, nooit erna.

Stap 4 — Events, herstart, `logs --previous`, `debug`

```
kubectl get events --sort-by=.lastTimestamp | tail -5 | cut -c1-120
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl exec $POD -- sh -c 'kill 1' ; sleep 5 ; kubectl get pod $POD
kubectl logs $POD --previous | tail -1
```

Observeer de laatste events van de namespace (`Killing`, `Scheduled`, `Pulled`, `Started`…); na `kill 1` dezelfde Pod met `RESTARTS 1`; en in de **vorige** logs `API arretee proprement.` : de laatste regel van de gestorven container.

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api --profile=general -- sh -c
'ps aux | head -3; wget -qO- localhost:8080/actuator/health'
```

Observeer na twee informatieregels van `kubectl`: `1 1000 … java -jar /app/api.jar` in `ps` — het proces van de API, gezien vanuit een `alpine`-container die niet in de image zit — en daarna `{"status":"UP"}`.

Uitleg. De Pod heeft het overleefd, de container is vervangen: `logs` zonder optie toont de nieuwe, `--previous` de oude. `kubectl debug --target` leent de container een shell en tools zonder er in de image te stoppen.

Stap 5 — RBAC: een account dat alleen mag lezen

```
kubectl apply -f rbac-reader.yaml
kubectl auth can-i get pods      --as=system:serviceaccount:lab16:reader
kubectl auth can-i delete pods   --as=system:serviceaccount:lab16:reader
kubectl auth can-i get secrets   --as=system:serviceaccount:lab16:reader
kubectl auth can-i get pods -n default --as=system:serviceaccount:lab16:reader
```

Observeer `yes`, `no`, `no`, `no`: de Pods van hier lezen, ja; ze verwijderen, de Secrets zien, of elders lezen, nee.

```
kubectl get pods --as=system:serviceaccount:lab16:reader | head -2
kubectl get secrets --as=system:serviceaccount:lab16:reader 2>&1 | cut -c1-140
kubectl auth can-i --list --as=system:serviceaccount:lab16:reader | head -6
```

Observeer de lijst van Pods die doorgaat; daarna `Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:lab16:reader" cannot list resource "secrets"…`; en in `--list`, `pods/log` en `pods` met `[get list watch]`, plus enkele generieke rechten (`selfsubjectreviews`, `/api`…) die elk geauthenticeerd subject heeft.

Uitleg. De Role zegt **wat** (`pods`, `pods/log`; `get list watch`), de RoleBinding zegt **wie** (`reader`), de namespace begrenst het geheel. `--as` vraagt de API server om te antwoorden "alsof": geen kubeconfig nodig om te testen.

Stap 6 — NetworkPolicy: de API sluiten

```
kubectl run client --image=docker.io/library/alpine --restart=Never --command -- sleep 600 ;
sleep 5
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo
kubectl apply -f netpol-api.yaml ; sleep 5
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo "exit=$?"
```

Observeer `{"status":"UP"}` vóór de policy; erna `wget: can't connect to remote host (10.43.x.x): Connection refused` en `exit=1`.

```
kubectl label pod client app=web ; sleep 3
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo
kubectl get networkpolicy
```

Observeer opnieuw `{"status":"UP"}`, zonder iets opnieuw te deployen: het label volstaat; en `api-only-from-web app=api`.

Uitleg. Zodra een policy `app=api` selecteert, komt alleen binnen wat ze opsomt. k3s past de policies toe (kube-router); dezelfde YAML op een CNI die ze negeert zou niets veranderen, en alleen deze test brengt dat aan het licht.

Opruimen

```
kubectl delete namespace lab16
kubectl config set-context --current --namespace=default
```

De namespace neemt Deployment, Service, Pods, LimitRange, quota, ServiceAccount, Role, RoleBinding en NetworkPolicy mee. De image `api:1.0` blijft in containerd voor labo 17.

Wat je nu moet kunnen beweren

- `Allocated resources` telt de requests; `kubectl top` meet het werkelijke verbruik; QoS `Burstable` wanneer requests ≠ limits.
- `OOMKilled`, `Exit Code 137`, `RESTARTS` die stijgt: de geheugenlimit; `Pending` + `Insufficient memory`: de request.
- LimitRange: standaardwaarden ingevoegd in een Pod zonder `resources`, weigering boven `max`; quota: `Used` / `Hard` per namespace.
- `logs --previous` voor de gestorven container, `debug --target` voor een shell naast een image zonder shell.
- `auth can-i … --as=system:serviceaccount:ns:naam`: yes/no, en `Forbidden` in het echt.
- NetworkPolicy: `Connection refused` zonder het label, `UP` ermee; toegepast door de CNI van k3s.